



Server Setup & Deployment Guide

Shaggy Dog Spa — Mobile Grooming Website

Hetzner CX33

Nuremberg (eu-central)

Ubuntu

from a Windows PC

Self-hosted · SQLite · local-disk media · no Vercel / Supabase / Docker

Manage everything online at `/admin` · hosts multiple websites on one box

Before you start

You'll need four things:

- A **Hetzner Cloud** account — `console.hetzner.cloud`
- Your **domain name** (with access to edit its DNS records)
- Your site's **GitHub repository URL** (push this project to GitHub first)
- Your **Windows PC** — everything below is done from PowerShell (built-in) + the browser

You already chose the server type and location: **CX33 (4 vCPU / 8 GB / 80 GB) in Nuremberg**. That's ~\$9.59/mo and works well for several low-traffic sites. (Nuremberg is in Germany, so pages load ~150 ms slower for US visitors than a US server would — harmless for a booking site, and your local Google ranking is unaffected.)

1 · Create your SSH key (on your Windows PC)

An SSH key is how your PC proves who it is to the server — safer than a password. In the Hetzner create-server screen this shows a ⚠ until you add one.

Open **PowerShell** on your PC and run:

```
ssh-keygen -t ed25519 -C "johnsyramos@gmail.com"
```

Press **Enter** through every prompt (accept the default file, no passphrase is fine to start). Then copy the *public* key to your clipboard:

```
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
```

Two keys are created. The `.pub` file is the *public* key you paste into Hetzner. The other file (no extension) is your *private* key — it stays on your PC, never share it.

2 · Finish creating the server

Back in the Hetzner create-server screen (Type = CX33, Location = Nuremberg, Image = Ubuntu):

- **SSH keys** → click **Add SSH key**, paste (Ctrl+V), name it `johns-pc`, Save. The ⚠ becomes a ✓.
- **Backups** → check it on (~\$2/mo). Automatic daily server snapshots — cheap insurance.
- **Name** → at the bottom, e.g. `shaggy-dog-spa`.
- **Create & Buy now** → wait ~30 seconds, then copy the server's **public IPv4 address** from the server list.

Leave Volumes, Firewalls, and Placement groups unchecked.

3 · First login

In PowerShell (replace with your server's IP):

```
ssh root@YOUR_SERVER_IP
```

Type `yes` at the fingerprint prompt. You're now on the server. Every command with `sudo` from here runs *on the server*.

4 · Create a non-root user

```
adduser deploy                # set a password when prompted
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy
```

Reconnect as that user (keep the root window open until this works):

```
ssh deploy@YOUR_SERVER_IP
```

5 · Firewall + swap

```
sudo apt update
sudo ufw allow OpenSSH
sudo ufw allow 80
sudo ufw allow 443
sudo ufw --force enable
```

Swap keeps the memory-hungry build from failing on the 8 GB box:

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h          # confirm swap shows 4Gi
```

6 · Install Node.js + tools

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -  
sudo apt-get install -y nodejs git sqlite3  
node -v          # should print v22.x
```

7 · Get the code + create the .env

```
cd ~  
git clone YOUR_GITHUB_REPO_URL shaggy-dog-spa  
cd shaggy-dog-spa
```

Create the `.env` — this keeps everything on SQLite + local disk, no external services and no CMS secret:

```
echo "DATABASE_URI=file:./shaggy.db"          > .env  
echo "NEXT_PUBLIC_SITE_URL=https://yourdomain.com" >> .env  
cat .env          # replace yourdomain.com with your real domain
```

8 · Install, build, and seed (first time only)

```
npm ci          # do NOT set NODE_ENV=production here – the build needs dev deps  
npm run build   # the memory-hungry step; swap (step 5) covers it  
npm run db:migrate # create the database schema  
npm run db:seed  # create the admin user + demo content. RUN ONCE.
```

⚠ **npm run db:seed** **wipes and recreates content** (and the media folder) each run. Run it once on first deploy. After that the owner edits content in `/admin` — never re-seed, or you'll erase their changes. (`db:migrate` is always safe to re-run.)

9 · Run it as a service (auto-start, auto-restart)

```
sudo tee /etc/systemd/system/shaggy.service >/dev/null <<'EOF'
[Unit]
Description=Shaggy Dog Spa website
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/home/deploy/shaggy-dog-spa
Environment=NODE_ENV=production
Environment=PORT=3000
ExecStart=/usr/bin/env npm run start
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now shaggy
sudo systemctl status shaggy      # should say "active (running)"
```

Live logs any time: `journalctl -u shaggy -f`

10 · Point your domain + automatic HTTPS (Caddy)

DNS — at your registrar, point the domain at the server's IPv4:

- A record, host `@` → `YOUR_SERVER_IP`
- A record, host `www` → `YOUR_SERVER_IP`

Install Caddy:

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr/share
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/sources
sudo apt update
sudo apt install -y caddy
```

Configure it (proxy your domain to the app on port 3000):

```
sudo tee /etc/caddy/Caddyfile >/dev/null <<'EOF'
yourdomain.com, www.yourdomain.com {
    reverse_proxy 127.0.0.1:3000
}
EOF
sudo systemctl reload caddy
```

Once DNS propagates, visit **<https://yourdomain.com>** — Caddy issues a free HTTPS certificate automatically.

11 · Log in and change the admin password

1. Go to **<https://yourdomain.com/admin>**
2. Log in: username `ShaggyDogSpa` · password `Admin2026!`
3. **Change the password immediately** (the seed login is public in the repo) at **Admin** → **Account**. It signs out any other device.

From here the client manages everything in the browser — content, prices, hours, photos (drag-and-drop "Replace Image"), reviews, and sees leads/bookings. No local software, no Docker.

12 · Backups (SQLite + media)

Everything that matters is one database file + the media folder:

```
mkdir -p ~/backups
crontab -e
```

Add this line (a nightly dated snapshot at 3am):

```
0 3 * * * cd /home/deploy/shaggy-dog-spa && sqlite3 shaggy.db ".backup '/home/deploy/backups/shagg
```

Also copy `~/backups` **off** the server (to your PC or a Hetzner Storage Box). A same-machine backup protects against mistakes, not against losing the machine.

13 · Updating the site later (done by you, not the client)

```
cd ~/shaggy-dog-spa
git pull
npm ci
npm run build
npm run db:migrate      # only needed if the update adds schema; always safe
sudo systemctl restart shaggy
```

Do **not** re-run `npm run db:seed` on updates — it would erase the owner's real content.

14 · Hosting more websites on the same server

This box can run several sites at once. One **Caddy** is the front door: it routes each domain to a different local port. Each site is fully isolated — its own folder, port, service, database, and media.

What must differ per site	Site A	Site B
Folder	<code>~/sites/shaggy</code>	<code>~/sites/clientb</code>
<code>PORT</code> (in its <code>.env</code>)	3000	3001
systemd service	<code>shaggy.service</code>	<code>clientb.service</code>
Caddy domain block	shaggydogspa.com	clientb.com
Database + media	its own	its own (automatic)

To add a site: repeat steps 7–11 in a new folder with a **unique** `PORT` , give it its **own systemd service** (a unique name), and add a domain block to the Caddyfile:

```
shaggydogspa.com, www.shaggydogspa.com {
    reverse_proxy 127.0.0.1:3000
}
clientb.com, www.clientb.com {
    reverse_proxy 127.0.0.1:3001
}
```

Then `sudo systemctl reload caddy` and point the new domain's DNS at the same server IP.

Memory — the one shared-box rule. A running site is light (~150–250 MB), so 8 GB handles several. But `npm run build` is heavy (up to an 8 GB heap), so **build/deploy one site at a time — never two builds at once**, and keep the 4 GB swap on. On 8 GB + 4 GB swap you can comfortably run about 3–6 low-traffic sites.

The full multi-site walkthrough (day-to-day ops, an all-sites backup cron) is in `MULTI-SITE.md` in the repo.

15 · Troubleshooting

Symptom	Check
Site won't load	<code>sudo systemctl status shaggy</code> · <code>journalctl -u shaggy -f</code>
HTTPS not working	<code>journalctl -u caddy -f</code> ; confirm DNS A records point to the server
Build "out of memory"	Confirm swap is on (<code>free -h</code>); build one site at a time
Images missing	They live on disk under <code>./media</code> ; re-seed once only if the DB is empty
Port already in use	Two sites share a <code>PORT</code> — give each a unique one
Caddy won't reload	<code>caddy validate --config /etc/caddy/Caddyfile</code> to find the error

What this setup does & does not need

-  **One Hetzner server** (runs 24/7) — the entire system.
-  **Client manages fully online** at `/admin` — content, prices, hours, photos, reviews, bookings.
-  **Multiple websites** on the one box, each isolated.
-  No Docker.  No Supabase.  No Vercel.  No external database or file storage.  No third-party CMS.
-  Lead/booking email alerts print to the server log; every lead & booking is still saved and visible in `/admin` . Add SMTP later if you want emailed alerts.